

数据结构与算法

第1章 绪论

张丽淼

光电信息获取与防护技术全国重点实验室

zhanglm@ahu.edu.cn

2026年3月6日

上节回顾

1、数据、数据元素、数据项、数据结构等基本概念



1.2 基本概念和术语

- 1、**数据** (data)—所有能输入到计算机中去的**描述客观事物的符号**
 - ◆ 数值性数据
 - ◆ 非数值性数据 (多媒体信息处理)
- 2、**数据元素** (data element) —数据的**基本单位**，也称**结点** (node) 或**记录** (record)
- 3、**数据项** (data item) —是构成数据元素的、有独立含义的数据**最小单位**，也称**域**(field)

三者之间的关系：数据 > 数据元素 > 数据项

例：学生表 > 个人记录 > 学号、姓名……

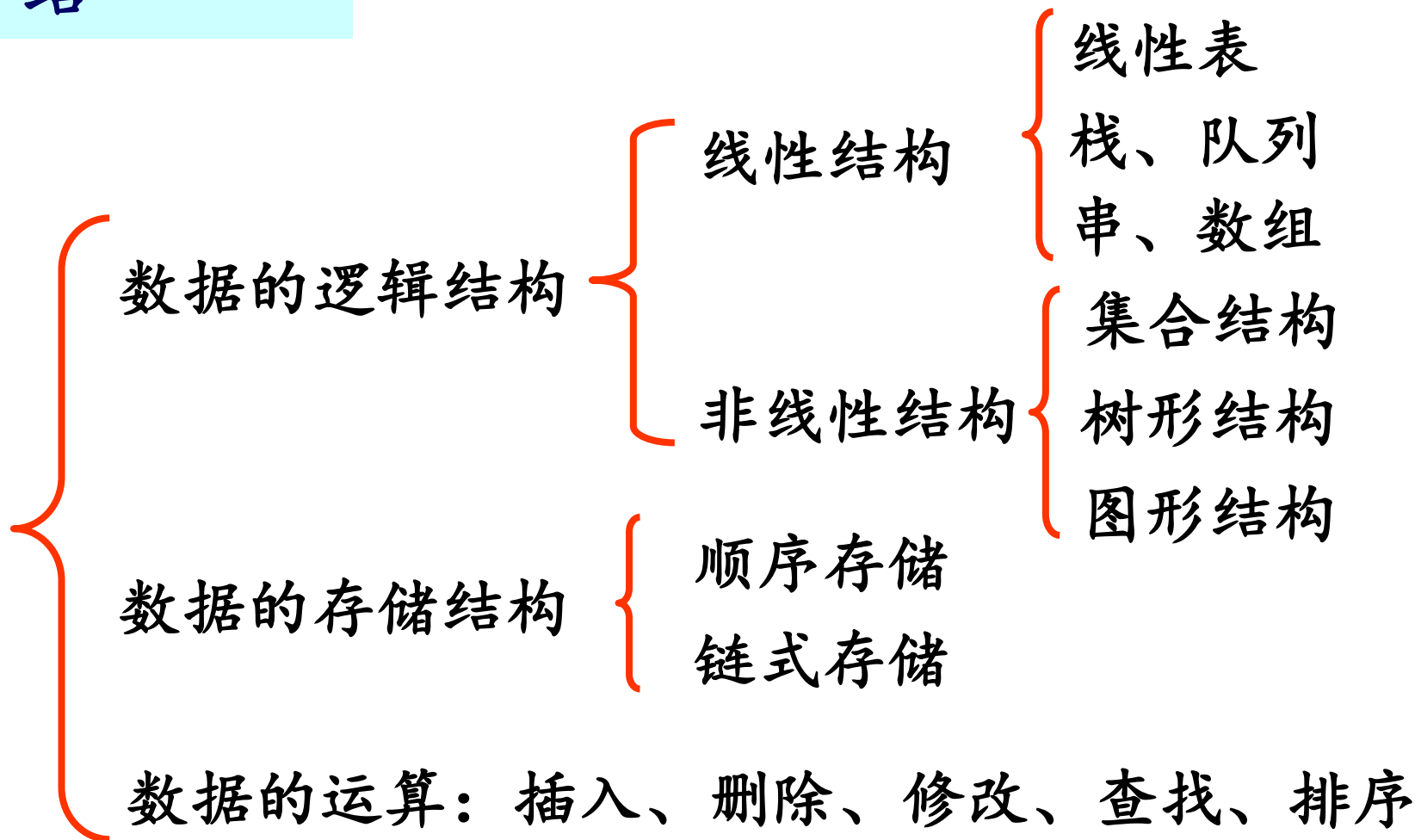
上节回顾

1、数据、数据元素、数据项、数据结构等基本概念

2、对数据结构的两个层次的理解

- 逻辑结构
- 存储结构

小结



运算的实现

依赖于

存储结构



存储结构

不唯一

逻辑结构

唯一

上节回顾

- 1、数据、数据元素、数据项、数据结构等基本概念
- 2、对数据结构的两个层次的理解
 - 逻辑结构
 - 存储结构
- 3、抽象数据类型的表示方法

抽象数据类型

(ADTs: Abstract Data Types)

指一个**数学模型**以及定义在该模型上的一组**操作**。

- ◆ 更高层次的数据抽象，不考虑计算机内的具体存储结构与运算的具体实现方法。
- ◆ 由用户定义，用以表示应用问题的**数据模型**
- ◆ 由**基本的数据类型**组成，并包括**一组相关的操作**
- ◆ **抽象性**：强调数据类型的数学特性，与其在计算机内部如何表示和实现无关。
- ◆ **可扩展性**：由已有的数据类型可扩展出尚未实现的数据类型。

抽象数据类型的定义格式

ADT 常用 定义 格式

ADT 抽象数据类型名{

数据对象：<数据对象的定义>

数据关系：<数据关系的定义>

基本操作：<基本操作的定义>

} ADT 抽象数据类型名

其中基本操作的定义格式为：

- 基本操作名（参数表）
- 初始条件：<初始条件描述>
- 操作结果：<操作结果表示>

注意：这只是一个抽象定义，还没有具体实现。

抽象数据类型的实现

注意：一个问题抽象为一个抽象数据类型后，仅仅是形式上的抽象定义，还没有达到问题解决的目的，要实现这个目标，就把抽象数据类型的定义转化为可以在计算机上实现的某种语言的代码，变为一个能用的具体的数据类型。

C语言实现抽象数据类型

- 用已有的数据类型定义描述它的存储结构
- 用函数定义描述它的操作



使用抽象数据类型

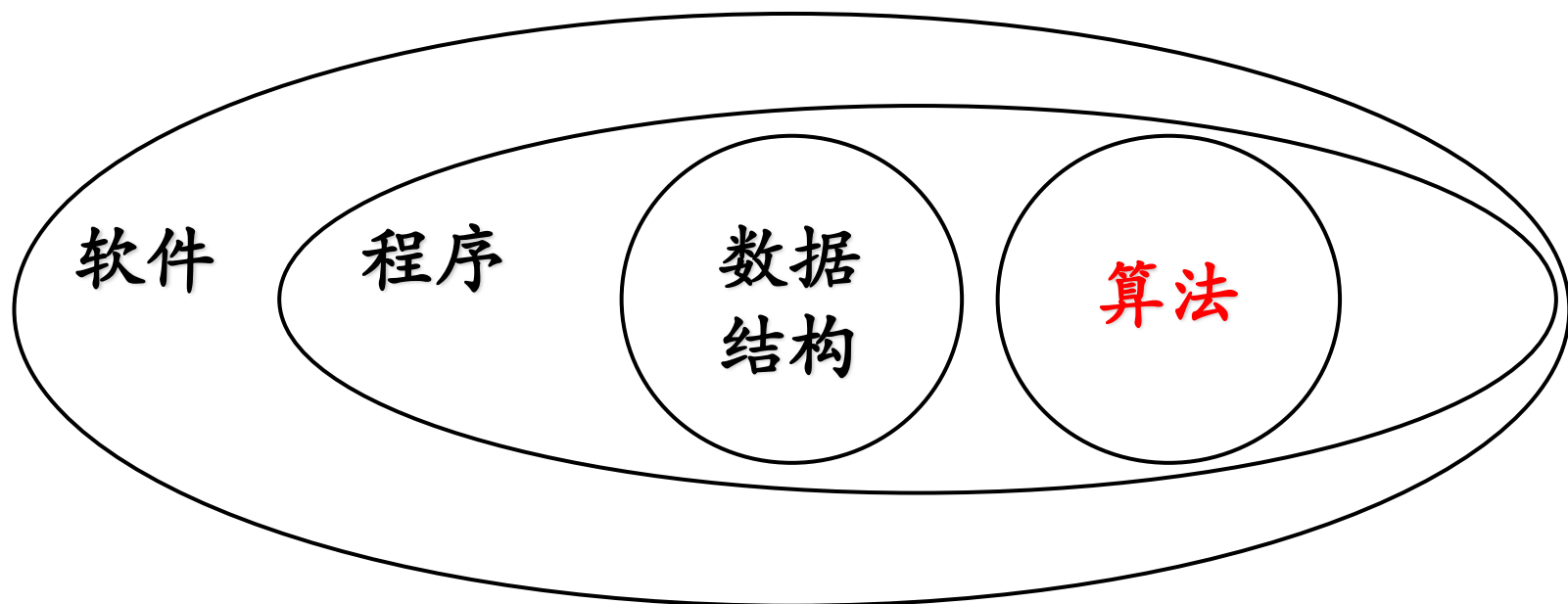
上节回顾

- 1、数据、数据元素、数据项、数据结构等基本概念
- 2、对数据结构的两个层次的理解
 - 逻辑结构
 - 存储结构
- 3、抽象数据类型的表示方法
- 4、算法和算法分析



1.4 算法和算法分析

算法的定义



- 算法定义：一个有穷的指令集，这些指令为解决某一特定任务规定了一个运算序列

更一般地，算法是“解决问题的处理步骤”。

算法的特性

算法的五个重要的特性

- (1) **有穷性**: 算法应在执行有穷步后结束, 能够停机。
 - (2) **确定性**: 每步定义都是确切、无歧义的。
 - (3) **可行性**: 可通过已实现基本运算有限次执行来实现, 也就是算法中每一个动作能够被机械地执行。
 - (4) **输入**: 0个或多个输入
 - (5) **输出**: 1个或多个输出
(处理结果)
- } 表示存在数据处理

算法的特性

➤ 算法和程序的区别

不一定满足有穷性

- 程序是指使用某种计算机语言对一个算法的具体实现，即具体要**怎么做**。
- 算法侧重于对解决问题的方法描述，即要**做什么**。

一定满足有穷性

算法用计算机语言描述 ⇨ 程序

算法描述

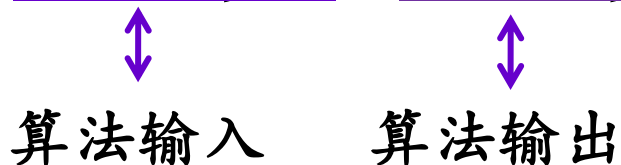


➤ 算法描述的一般格式

```
返回值 算法对应的函数名(形参列表)
{ //临时变量的定义
  //实现由输入参数到输出参数的操作
  ...
}
```

} 函数体

- 返回值：通常为bool类型，表示算法是否成功执行。
- 形参列表：由输入型参数和输出型参数构成。



算法描述

➤ 如何描述输出型参数？

C++语言中提供了一种引用运算符“&”用于描述输出型参数。

引用示例：

```
int a=10;
```

```
int &b=a;
```

a
b

10

- 定义变量b是变量a的引用变量
- 两个变量共享内存空间, b = 10
- 效果：a、b同步发生改变

➤ 为什么要引用？

- ✓ 普通参数传递是单向值传递：形参改变不影响实参
- ✓ 引用参数实现双向传递：形参实参同步变化

算法描述

示例：设计一个交换两个整数的算法。

编写一个函数swap1(x, y):

```
void swap1(int x, int y)
{
    int tmp;
    tmp=x; x=y; y=tmp;
}
```

} 交换形参x和y的值



当执行语句swap1(a, b)时，a和b实参值不会发生交换。

```
int a = 3, b = 5;
swap1(a, b);
printf("a = %d, b = %d\n", a, b); // 输出结果: a = 3, b = 5
```

分析：x、y既是输入型参数，也是输出型参数

算法描述

要真正修改实参的值，我们需要知道实参在内存中的地址，然后通过地址直接访问那块内存。C语言中，指针就是用来存储地址的变量。

改正方法1：采用指针的方式来传形参的值，需将上述函数改为：

```
void swap2(int *x, int *y)
{
    int tmp;
    tmp=*x;           //将x的值放在tmp中
    *x=*y;             //将x所指的值改为*y
    *y=tmp;            //将y所指的值改为tmp
}
```

} 交换形参x和y所指向
的值

上述函数的调用改为swap2(&a, &b) ⇒ 比较复杂。

```
int a = 3, b = 5;
swap2(&a, &b);    // 传入a和b的地址
printf("a = %d, b = %d\n", a, b); // 输出: a = 5, b = 3
```

算法描述

改正方法2：采用引用型形参 $\Rightarrow$ 将输出型形参改为引用类型。（C++）

```
void swap(int &x, int &y)
```

```
//形参前的“&”符号不是指针运算符
```

```
{ int tmp=x;
```

```
    x=y; y=tmp;
```

```
}
```

} 交换形参x和y的值

当执行语句swap(*a*, *b*)时，形、实参的匹配相当于：

int &x=a; //x为a的引用，x和a共享同一块内存

int &y=b; //y为b的引用，y和b共享同一块内存

这样，*a*与x共享存储空间、*b*与y共享存储空间，因此执行函数后*a*和*b*的值发生了交换 $\Rightarrow$ 简单明了。

算法描述

➤ 普通的参数传递

```
void fun1(int n)
{ int m=2;
  fun2(m);
  printf("%d\n", m);
}
```

实参



```
void fun2(int x)
{ x++;
  printf("%d\n", x);
}
```

普通形参



算法描述

➤ 引用类型的参数传递

```
void fun1(int n)
```

```
{ int m=2;          ↗ 实参
```

```
  fun2(m);
```

```
  printf(“%d\n”, m);
```

```
}
```



```
void fun2(int &x)
```

```
{ x++;              ↖ 引用型形参
```

```
  printf(“%d\n”, x);
```

```
}
```

① 实参到形参单向值传递



② 形参回传给实参，实参
和形参同步发生改变

算法描述

【例1.5】设计一个算法：求一元二次方程 $ax^2+bx+c=0$ 的根。

算法可以采用自然语言、流程图或者表格方式等来描述。

但是，一个学习计算机的学生应该使用某种计算机语言来描述算法。本课程采用C/C++语言描述算法。

C++的作用是在描述算法时使用其提供的引用类型！

算法描述

解

输入: $a\ b\ c$

solution

输出:

● 根个数

● $x_1\ x_2$

```
int solution(float a, float b, float c,  
             float &x1, float &x2)  
{ double d;  
  d=b*b-4*a*c;  
  if (d>0)  
  { x1=(-b+sqrt(d))/(2*a);  
    x2=(-b-sqrt(d))/(2*a);  
    return 2;      //2个实根  
  }  
  else if (d==0)  
  { x1=(-b)/(2*a);  
    return 1;      //1个实根  
  }  
  else              //d<0的情况  
  { return 0;      //不存在实根  
  }  
}
```

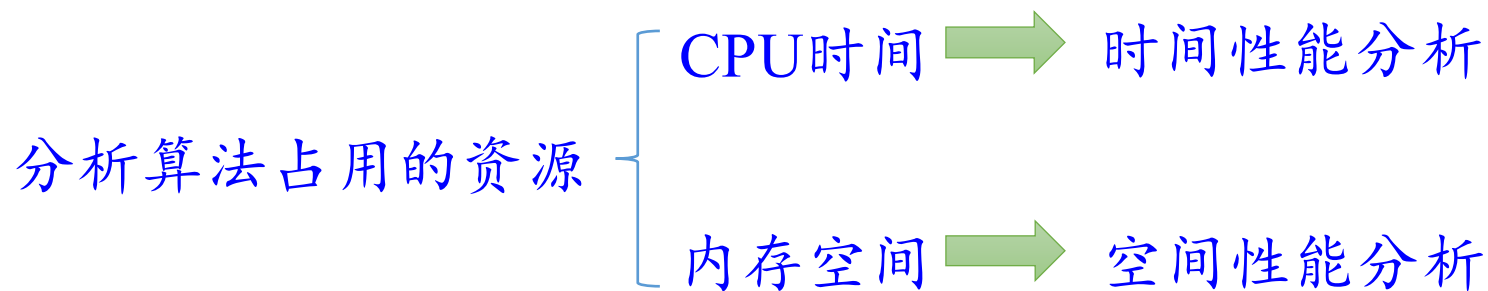
算法的评价

通常设计一个“好”的算法应达到以下目标：

- ◆ **正确性**：能正确解决问题
- ◆ **可读性**：思路清晰，易于理解
- ◆ **健壮性**：对非法输入能恰当处理，不会崩溃
当输入的**数据非法**时，算法应恰当**作出反应或进行相应处理**，而不是产生莫名其妙的输出结果。
不应中断执行，二是返回一个**表示错误或错误性质的值**，以便在更高层次上处理。
- ◆ **高效性（时间代价和空间代价小）**
时间复杂度和空间复杂度是衡量算法的两个主要指标。两者都与**问题规模**有关。

算法分析

一个好的算法首先要具备正确性、健壮性、可读性，在这几个方面都满足的条件下，主要考虑算法的效率，通过算法的效率高低来评判不同算法的优劣程度。



算法分析目的：分析算法的时空效率以便改进算法性能。

算法分析方式

通常有两种度量算法效率的方法。

1.事后统计：编写算法对应程序，利用计算机内的计时功能，统计其执行时间。

```
1  time_t start,end;  
2  start =time(NULL);//or time(&start);  
3  //...calculating...  
4  end =time(NULL);  
5  printf("time=%d\n",difftime(end,start));
```

```
1  clock_t start,end;  
2  start = clock();  
3  //...calculating...  
4  end = clock();  
5  printf("time=%f\n",(double)end-start)/CLK_TCK);
```

算法分析方式

通常有两种度量算法效率的方法。

1.事后统计：编写算法对应程序，利用计算机内的计时功能，统计其执行时间。

缺点：

①必须先运行依据算法编制的程序

②所得时间统计量依赖于硬件、软件等环境因素，掩盖算法本身的优劣

算法分析方式

一个高级语言程序在计算机上运行所消耗的时间**取决于**：

- ①依据的算法选用何种策略
- ②问题的规模
- ③程序语言
- ④编译程序产生机器代码质量
- ⑤机器执行指令速度

同一个算法用**不同的语言**、**不同的编译程序**、在**不同的计算机**上运行，效率均不同——使用**绝对时间单位**衡量算法效率不合适

2.事前估算：撇开上述因素，认为算法的执行时间是问题规模 n 的函数。

一个特定的算法“运行工作量”的大小，只依赖于问题的规模(通常用整数量 n 来表示)，或者说，他是问题规模 n 的函数。

- ① 列表长度
- ② 矩阵阶数
- ③ 多项式项数
- ④ 集合元素个数
- ⑤ 字符数量，等等

一个特定的算法“运行工作量”的大小，只依赖于问题的规模(通常用整数量 n 来表示)，或者说，他是问题规模 n 的函数。



“运行工作量”如何度量？

等于算法中所有基本语句的执行次数（语句频度）之和，是问题规模 n 的某个函数，记为 $T(n)$ 。

（假设执行每条语句所需的时间均为单位时间，与算法好坏无关）

算法的时间复杂度分析

如何估算算法的时间复杂度？

算法的时间特性用执行基本操作的次数 $T(n)$ 来度量。

```
for ( i=1; i <= n; i ++ )  
    for (j = 1; j <=n; j++ )  
    {  
        c[i][j]=0;  
        for (k= 1; k<=n; k++ )  
            c[i][j]= c[i][j]+a[i][k]*b[k][j];  
    }
```

算法的时间复杂度分析

如何估算算法的时间复杂度？

算法的时间特性用执行基本操作的次数 $T(n)$ 来度量。

```
for ( i=1; i <= n; i ++ )     $n+1$ 
    for (j = 1; j <=n; j++ )   $n*(n+1)$ 
    {
        c[i][j]=0;             $n^2$ 
        for (k= 1; k<=n; k++ )  $n^2*(n+1)$ 
            c[i][j]= c[i][j]+a[i][k]*b[k][j];  $n^3$ 
    }
```

$$T(n)=2n^3+3n^2+2n+1$$

算法的时间复杂度分析

如何估算算法的**时间复杂度**？

算法的时间特性用**执行基本操作的次数** $T(n)$ 来度量。

```
for ( i=1; i <= n; i++)
```

$n+1$

```
for ( j=1; j <= i; j++)
```

$n*(n+1)$

- ◆ 算法中重复执行次数和算法的执行时间成正比的语句

n^2

- ◆ 对算法运行时间的贡献最大

```
for ( k=1; k <= n; k++)
```

$n^2*(n+1)$

```
c[i][j] = c[i][j] + a[i][k]*b[k][j];
```

n^3

```
}
```

$$T(n) = 2n^3 + 3n^2 + 2n + 1$$

算法的时间复杂度分析

如何估算算法的时间复杂度？

算法的时间特性用执行基本操作的次数 $T(n)$ 来度量。

```
for ( i=1; i <= n; i++)      n+1
    for ( j=1; j <= n; j++)  n*(n+1)
        c[i][j] = c[i][j] + 1;
```

◆ 算法中重复执行次数和算法的执行时间成正比的语句

◆ 对算法运行时间的贡献最大

n 越大算法的执行时间越长

◆ 排序： n 为记录数

◆ 矩阵： n 为矩阵的阶数

◆ 多项式： n 为多项式的项数

◆ 集合： n 为元素个数

◆ 树： n 为树的结点个数

◆ 图： n 为图的顶点数或边数

$$T(n) = 2n^3 + 3n^2 + 2n + 1$$

算法时间复杂度的渐进表示方法

- 为了便于比较不同算法的时间效率。我们进比较他们的数量级，即渐进时间复杂度。

例如，两个不同的算法的时间消耗分别是：

$$T_1(n) = 10n^2 \quad T_2(n) = 5n^3 + 10$$

算法时间复杂度的渐进表示方法

- 为了便于比较不同算法的时间效率。我们进比较他们的数量级，即渐进时间复杂度。

例如，两个不同的算法的时间消耗分别是：

$$T_1(n) = 10n^2 \quad T_2(n) = 5n^3 + 10$$

- 若有某个辅助函数 $f(n)$ ，使得当 n 趋近于无穷大时， $T(n)/f(n)$ 的极限值为不等于零的常数 c ，则称 $f(n)$ 是 $T(n)$ 的同数量级函数，记作 **$T(n) = O(f(n))$** 。

$$f_1(n) = n^2 \quad f_2(n) = n^3 \quad \lim_{n \rightarrow \infty} \frac{T(n)}{f(n)} = c$$

- 称 $O(f(n))$ 为算法的**渐进时间复杂度**，简称复杂度，其中 O 是数量级的符号。

算法时间复杂度的渐进表示方法

- 算法的（渐近）时间复杂度：

$$T(n)=O(f(n))$$

表示随着n的增大，算法执行的时间的增长率和f(n)的增长率相同。

如何理解数学符号“O”？

算法时间复杂度的渐进表示方法

• 算法的（渐近）时间复杂度：

$$T(n)=O(f(n))$$

表示随着n的增大，算法执行的时间的增长率和f(n)的增长率相同。

如何理解数学符号“O”？

基于两个函数的**比率求极限**（也即输入规模n无限大），有三种极限情况发生：

$$\lim_{n \rightarrow \infty} \frac{t(n)}{g(n)} \begin{cases} 0 & \text{表明} t(n) \text{的增长次数比} g(n) \text{小} \\ c & \text{表明} t(n) \text{的增长次数和} g(n) \text{相同} \\ \infty & \text{表明} t(n) \text{的增长次数比} g(n) \text{大} \end{cases}$$

算法时间复杂度的渐进表示方法

• 算法的（渐近）时间复杂度：

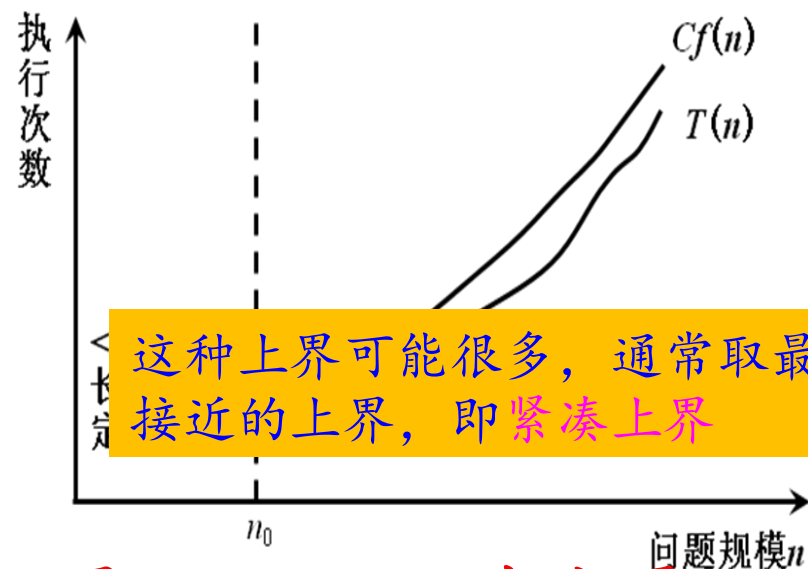
$$T(n) = O(f(n))$$

表示随着 n 的增大，算法执行的时间的增长率和 $f(n)$ 的增长率相同。

如何理解数学符号“ O ”？

数学符号“ O ”的定义为：

若 $T(n)$ 和 $f(n)$ 是定义在正整数集合上的两个函数，则 O 表示为 $T(n)$ 找到了一个上界 $f(n)$ ，即存在正的常数 C 和 n_0 ，使得当 $n \geq n_0$ 时都满足 $0 \leq T(n) \leq Cf(n)$ 。



$T(n) = O(f(n))$ 表示的含义是 $T(n)$ 以 $f(n)$ 为上界

算法时间复杂度的渐进表示方法

- 算法的（渐近）时间复杂度：

$$T(n)=O(f(n))$$

表示随着 n 的增大，算法执行的时间的增长率和 $f(n)$ 的增长率相同。

本质上讲，是一种
 $T(n)$ 最高数量级的比较

也就是只求出 $T(n)$ 的最高阶，忽略其低阶项和常系数，这样既可简化 $T(n)$ 的计算，又能比较客观地反映出当 n 很大时算法的时间性能。

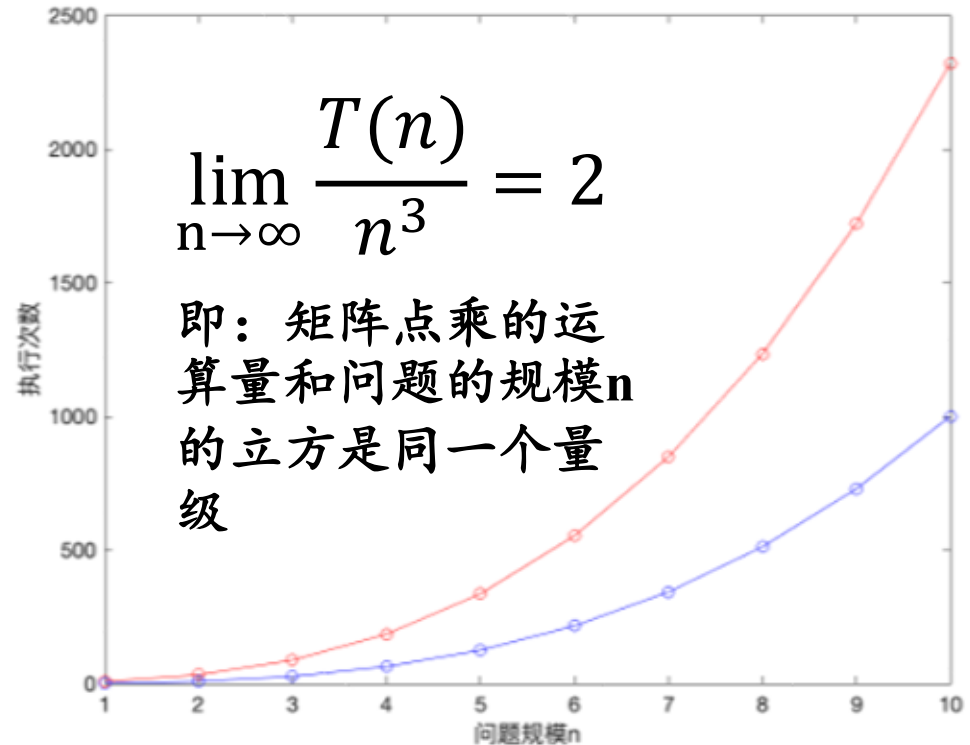
$$\text{例如： } T(n) = 2n^2 + 2n + 1 = O(n^2)$$

$$T(n)=2n^3+3n^2+2n+1$$

```

for ( i=1; i <= n; i ++ )
    for ( j = 1; j <=n; j-
    {
        c[i][j]=0;
        for (k= 1; k<=n; k++)
            c[i][j]= c[i][j]+
    }

```

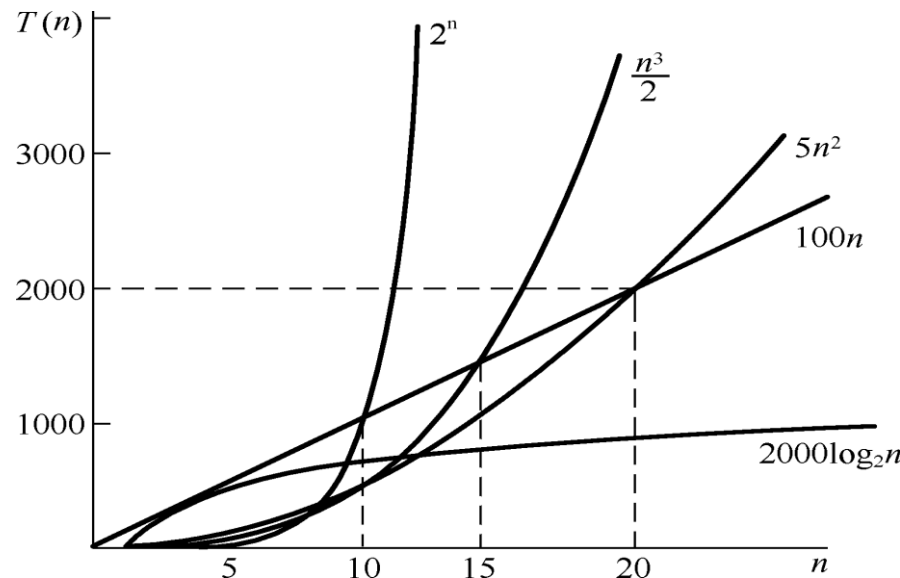


$$T(n)=O(f(n))=O(n^3)$$

一般地：

- 一个没有循环的算法的执行时间与问题规模 n 无关，记作 $O(1)$ ，也称作**常数阶**。
- 一个只有一重循环的算法的执行时间与问题规模 n 的增长呈线性增大关系，记作 $O(n)$ ，也称**线性阶**。
- 其余常用的算法时间复杂度还有**平方阶** $O(n^2)$ 、**立方阶** $O(n^3)$ 、**对数阶** $O(\log_2 n)$ 、**指数阶** $O(2^n)$ 等。

当n取得很大时，指数时间算法和多项式时间算法在所需时间上非常悬殊



时间复杂度 $T(n)$ 按数量级递增顺序为：

复杂度低

复杂度高

常数阶	对数阶	线性阶	线性对数阶	平方阶	立方阶	...	K次方阶	指数阶
$O(1)$	$O(\log_2 n)$	$O(n)$	$O(n \log_2 n)$	$O(n^2)$	$O(n^3)$		$O(n^k)$	$O(2^n)$

$O(n!)$

多项式时间 (P)

指数时间 (NP)

常数阶	对数阶	线性阶	线性对数阶	平方阶	立方阶	...	K次方阶	指数阶
$O(1)$	$O(\log_2 n)$	$O(n)$	$O(n \log_2 n)$	$O(n^2)$	$O(n^3)$		$O(n^k)$	$O(2^n)$

算法时间性能比较：假如求同一问题有两个算法：A和B，如果算法A的平均时间复杂度为 $O(n)$ ，而算法B的平均时间复杂度为 $O(n^2)$ 。

一般情况下，认为算法A的时间性能好比算法B。

示例

某算法的时间复杂度为 $O(n^2)$ ，表明该算法的（ ）。

A. 问题规模是 n^2

B. 执行时间等于 n^2

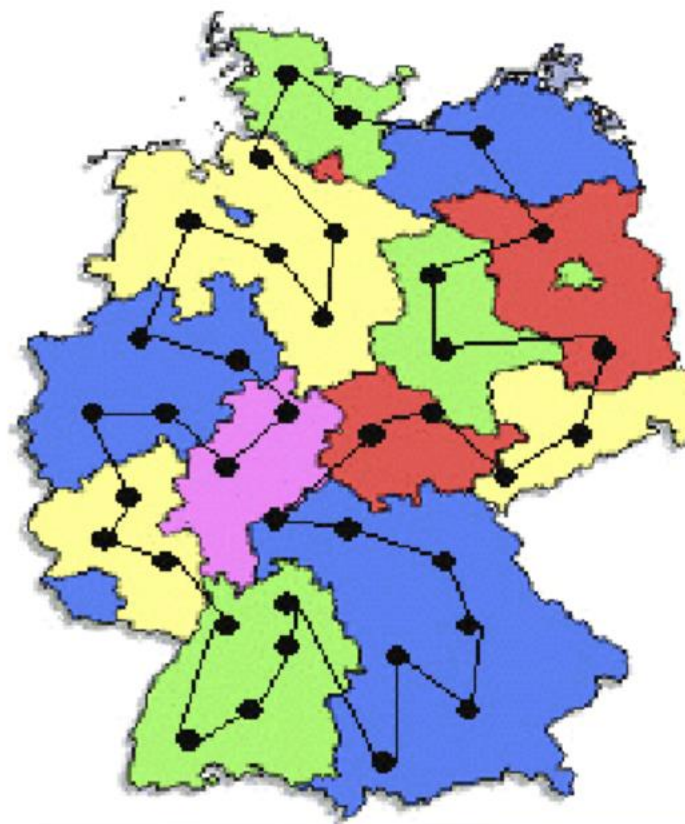
C. 执行时间与 n^2 成正比

D. 问题规模与 n^2 成正比

答：C。

TSP旅行商问题(NP问题)

旅行家要旅行 n 个城市，要求经历各个城市且仅经历一次，然后回到出发城市，并要求所走的路程最短。



枚举法的时间复杂度为: $O(n!)$

分析算法时间复杂度的基本方法

- 找出**语句频度最大**的那条语句作为**基本语句**
- 计算**基本语句**的频度得到问题规模 n 的某个函数 $T(n)$
- 取其数量级用符号“ O ”表示

```
x = 0; y = 0;
```

```
for ( int k = 0; k < n; k ++ )  
    x ++;
```

```
for ( int i = 0; i < n; i ++ )  
    for ( int j = 0; j < n; j ++ )  
        y ++;
```

分析算法时间复杂度的基本方法

- 找出**语句频度最大**的那条语句作为**基本语句**
- 计算**基本语句**的频度得到问题规模 n 的某个函数 $T(n)$
- 取其数量级用符号“ O ”表示

```
x = 0; y = 0;
```

```
for ( int k = 0; k < n; k ++ )  
    x ++;
```

```
for ( int i = 0; i < n; i ++ )  
    for ( int j = 0; j < n; j ++ )  
        y ++;
```

$$T(n) = O(n^2)$$

$$T(n) = n(n+1)$$

分析算法时间复杂度的基本方法

$$T(n)=2n^3+3n^2+2n+1 \Rightarrow T(n)=O(f(n))= O(n^3)$$

下面是总结的大O表示原则：

1. 用常数1取代运行次数函数中的所有加法常数。
2. 在修改后的运行次数函数中，只保留最高阶项。
3. 如果最高阶项存在且不是1，则去除与这个项目相乘的常数。得到的结果就是大O阶。

简化的算法时间复杂度分析

```
void exam ( float x[ ][ ], int m, int n ) {  
    float sum [ ];  
    for ( int i = 0; i < m; i++ ) {  
        sum[i] = 0.0;  
        for ( int j = 0; j < n; j++ )  
            sum[i] += x[i][j];  
    }  
    for ( i = 0; i < m; i++ )  
        cout << i << “ : ” << sum [i] << endl;  
}
```

时间复杂度是由**嵌套最深层**语句的频度决定的

简化的算法时间复杂度分析

```
void exam ( float x[ ][ ], int m, int n ) {  
    float sum [ ];  
    for ( int i = 0; i < m; i++ ) {  
        sum[i] = 0.0;  
        for ( int j = 0; j < n; j++ )  
            sum[i] += x[i][j]; •  
    }  
    for ( i = 0; i < m; i++ )  
        cout << i << “ : ” << sum [i] << endl;  
}
```

$T(n) = O(m * n)$

$f(n) = m * n$

时间复杂度是由**嵌套最深层**语句的频度决定的

例1: $N \times N$ 矩阵相乘

```
for(i=1;i<=n;i++)  
  for(j=1;j<=n;j++)  
    {c[i][j]=0;  
     for(k=1;k<=n;k++)  
       c[i][j]=c[i][j]+a[i][k]*b[k][j];  
    }
```

 $T(n) = O(n^3)$

算法中的基本操作语句为 $c[i][j]=c[i][j]+a[i][k]*b[k][j];$

$$T(n) = \sum_{i=1}^n \sum_{j=1}^n \sum_{k=1}^n 1 = \sum_{i=1}^n \sum_{j=1}^n n = \sum_{i=1}^n n^2 = n^3 = o(n^3)$$

例2:

```
for( i=1; i<=n; i++)
```

```
    for (j=1; j<=i; j++)
```

```
        for (k=1; k<=j; k++)
```

```
            x=x+1;
```

$$\text{语句频度} = \sum_{i=1}^n \sum_{j=1}^i \sum_{k=1}^j 1 = \sum_{i=1}^n \sum_{j=1}^i j = \sum_{i=1}^n \frac{i(i+1)}{2}$$

例2:

```
for( i=1; i<=n; i++)
```

```
    for (j=1; j<=i; j++)
```

```
        for (k=1; k<=j; k++)
```

```
            x=x+1;
```

$$\begin{aligned}\text{语句频度} &= \sum_{i=1}^n \sum_{j=1}^i \sum_{k=1}^j 1 = \sum_{i=1}^n \sum_{j=1}^i j = \sum_{i=1}^n \frac{i(i+1)}{2} \\ &= \frac{1}{2} \left(\sum_{i=1}^n i^2 + \sum_{i=1}^n i \right) \\ &= \frac{1}{2} \left(\frac{n(n+1)(2n+1)}{6} + \frac{n(n+1)}{2} \right) \\ &= \frac{n(n+1)(n+2)}{6}\end{aligned}$$

例2:

```
for( i=1; i<=n; i++)  
    for (j=1; j<=i; j++)  
        for (k=1; k<=j; k++)  
            x=x+1;
```

定理1.1

若 $f(n)=a_m n^m + a_{m-1} n^{m-1} + \dots + a_1 n + a_0$
是 m 次多项式, 则 $T(n)=O(n^m)$ 。

忽略所有低次幂项和
最高次幂系数, 体现
出增长率的含义

$$\begin{aligned}\text{语句频度} &= \sum_{i=1}^n \sum_{j=1}^i \sum_{k=1}^j 1 = \sum_{i=1}^n \sum_{j=1}^i j = \sum_{i=1}^n \frac{i(i+1)}{2} \\ &= \frac{1}{2} \left(\sum_{i=1}^n i^2 + \sum_{i=1}^n i \right) \\ &= \frac{1}{2} \left(\frac{n(n+1)(2n+1)}{6} + \frac{n(n+1)}{2} \right) \\ &= \frac{n(n+1)(n+2)}{6}\end{aligned}$$

例3：分析以下程序段的时间复杂度

`i=1;`

①

`while(i<=n)`

`i=i*2;`

②

关键是找出来执行次数与n的关系，并表示称n的函数

若循环执行1次： $i=1*2=2$,

若循环执行2次： $i=2*2=2^2$,

若循环执行3次： $i=2*2=2^3$,,

若循环执行x次： $i=2^x$

设语句②执行次数为x次，由循环条件

$i \leq n$, $\therefore 2^x \leq n \therefore x \leq \log_2 n$

即 $T(n) \leq \log_2 n$, 取最大值 $T(n) = \log_2 n$

所以该程序段的时间复杂度 $T(n) = O(\log_2 n)$

算法的最好、最坏、平均时间复杂度

有的情况下，算法中基本操作重复执行的次数还随问题的**输入数据集**不同而不同

例4：顺序查找，在数组a[i]中查找值等于e的元素，返回其所在位置。

```
for (i=0;i< n;i++)  
    if (a[i]==e) return i+1;  
return 0;
```

- 最好情况：1次
- 最坏情况：n
- 平均时间复杂度为： $O(n)$

算法的最好、最坏、平均时间复杂度

定义： 设一个算法的输入规模为 n ， D_n 是所有输入的集合，任一输入 $I \in D_n$ ， $P(I)$ 是 I 出现的概率，有 $\sum_{I \in D_n} P(I) = 1$ ， $T(I)$ 是算法在输入 I 下的执行时间，则算法的**平均时间复杂度**为：

$$E(n) = \sum_{I \in D_n} P(I) \times T(I)$$

算法的最好、最坏、平均时间复杂度

例如，10个1~10的整数序列递增排序 ($n=10$)：

$$I_1 = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$$

$$I_2 = \{2, 1, 3, 4, 5, 6, 7, 8, 9, 10\}$$

...

$$I_m = \{10, 9, 8, 7, 6, 5, 4, 3, 2, 1\}$$

构成 D_n

所有可能的初始序列有 m 个， $m=10!$ $\Rightarrow P(I)=1/m$

算法的最好、最坏、平均时间复杂度

算法的**最坏时间复杂度**为： $W(n)=\underset{I \in D_n}{\text{MAX}}\{T(n)\}$

算法的**最好时间复杂度**为： $B(n)=\underset{I \in D_n}{\text{MIN}}\{T(n)\}$

请课下思考书P21 例1.8

算法的空间复杂度分析——渐进空间复杂度

■ 算法要占据的空间

- 算法本身要占据的空间，输入/输出，指令，常数，变量等
- 算法要使用的**辅助空间**

空间复杂度：用于量度一个算法运行过程中**临时占用的存储空间**大小。

一般也作为问题规模 n 的函数，采用数量级形式描述，记作：

$$S(n) = O(g(n))$$

算法的空间复杂度分析——渐进空间复杂度

例5：将一维数组a中的n个数逆序存放到原数组中。

$$S(n) = O(1)$$

原地工作

【算法1】

```
for(i=0;i<n/2;i++)  
{  
    t=a[i];  
    a[i]=a[n-i-1];  
    a[n-i-1]=t;  
}
```

$$S(n) = O(n)$$

【算法2】

```
for(i=0;i<n;i++)  
    b[i]=a[n-i-1];  
for(i=0;i<n;i++)  
    a[i]=b[i];
```

若一个算法的空间复杂度为 $O(1)$ ，则称此算法为原地工作或就地工作算法。